



SQL Report

Database Design of a Banking System

Andreas Poulsen
Torkil Sørin Jógvansson



NVDRit
2026-03

NÁTTÚRUVÍSINDATEILD
Faculty of Science and Technology

FRÓÐSKAPARSETUR FØROYA
University of the Faroe Islands

Tittulblað

Heiti / Title

Databasu Design av einari Bankaskipan

Database Design of a Banking System

Høvundar / Authors

Andreas Poulsen, Torkil Sørin Jógvansson

Vegleiðari / Supervisor

Hans Blaasvær

Ábyrgd vegleiðari
Responsible Supervisor

/ Hans Blaasvær, Fróðskaparsetur Føroya

Ritslag / Report Type

Rapport, KT-Verkfrøði

Report, Software Engineering

Latíð inn / Submitted

27. mars 2026

NVD Rit

2026:03

©

Náttúruvísindadeildin og høvundarnir

ISSN

1601-9741

Útgevur / Publisher

Náttúruvísindadeildin, Fróðskaparsetur Føroya

Bústaður / Address

Vestara Bryggja 15, FO 100 Tórshavn,
Føroyar (Faroe Islands)

+298 352 550 • +298 352 551 • nvd@setur.fo

Contents

1	Abstract	2
2	Problem Statement	2
3	Introduction	2
4	Database Design	3
5	Implementation of the Database	5
5.1	The GUI	11
6	Discussion	13
7	Conclusion	14
8	Appendix	14

1 Abstract

This report concerns the design and implementation of a banking database in Oracle SQL. The purpose of the project was to create a system that could model important parts of a banking environment, including persons, customers, employees, accounts, drafts, bookings, and interest.

We first go through the database design, including the ER diagram, cardinalities, keys, constraints, and normalization. After this, it moves on to the implementation, where tables, views, triggers, functions, and stored procedures are used to build the final system. Special focus is given to functionality such as p-tal generation, account number generation, draft and booking handling, spouse and parent views to accounts and aggregate views.

The result is a working database solution that covers the main requirements of the assignment and shows both the theoretical design and the practical implementation of a banking system.

2 Problem Statement

We were given the task of making a banking system with some specific requirements that had to be fulfilled. One of the main requirements was to design and implement an ER diagram with mapped cardinalities that matches a banking database in Oracle SQL.

The system had to support creation of persons, including generation of p-tal, as well as accounts, drafts, bookings, and interest handling. It also had to support visibility rules where parents can see their children's accounts, and married couples can see each other's accounts.

Another important part of the task was that the solution should be implemented in Oracle SQL with relevant database functionality such as tables, constraints, views, triggers, stored procedures, and functions.

In this report, we describe the problem, the database design, and how the different requirements were implemented in the final system.

3 Introduction

In this report, we go through the design process behind our banking database and explain the main ideas that shaped the system. We describe the design phase and look at the logic behind the database, including cardinalities, keys, and normalization.

After this theoretical part, we move on to the practical implementation of the database in Oracle SQL. Here we show how the system was built in practice and how the different parts work together in the final solution.

We also explain how we solved the different parts of the assignment and give our reasoning for the solutions we chose. Where relevant, we also mention alternative solutions that could have been used.

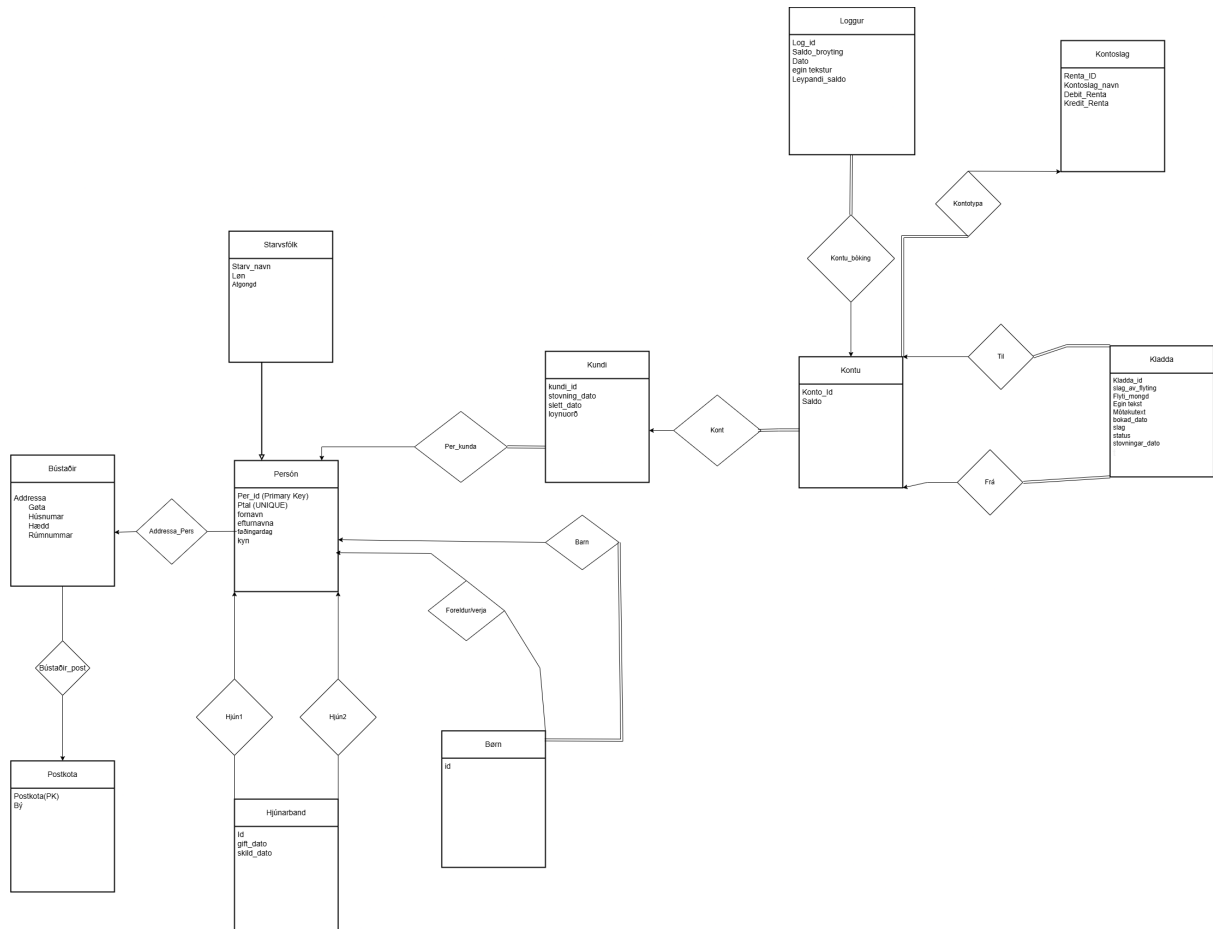


Figure 1: This is the entity relation diagram showing the relationships between the different tables.

In this way, the report covers both the theoretical database design and the practical implementation of the final system, and shows how the different requirements were translated into a working banking database.

4 Database Design

In this section we are going to be looking at the design of our database and our ER-diagram. Figure 1 shows us our ER-diagram. We will be describing the most important relationships here. We have a one to one relationship between person and customer, where every customer is a person but not vice versa. Between customer and account we have a one to many relationship with full participation on the account side. The same relationship is between account and loggur, and account and account type. The final two relationships we will highlight are the familial relationships used to control views for accounts owned by a spouse or child. The person table has two one to many relationships with the marriage table to account for two people being in the same marriage. There is also a similar relationship between the person table and children where a child has a guardian/parent and the child is also a person itself.

We have multiple constraints that the database needs to uphold.

- Every person needs a personal number called a p-tal that is unique and fulfills the requirements of a p-tal.
- Every person needs a valid birthday and gender in order to get a valid p-tal
- Every account the database contains needs to be generated in accordance with the account number standard.
- Every booking draft needs to have a positive amount of money moved. In other words you can't forcibly take money from someone else. A booking has to be of a valid type and has to have a status
- For marriage and child tables we have constrained the marriage to disallow marriage to yourself and you can't be both a child and the guardian of yourself.

We are going to be looking at some of the functional dependencies of our tables. The table person has two super keys.

$$Per_id \longrightarrow fingardag$$

$$Per_id \longrightarrow kyn$$

$$Per_id \longrightarrow fornavn$$

$$Per_id \longrightarrow efturnavn$$

$$Per_id, fingardag, kyn \longrightarrow Ptal$$

Since Ptal is also a unique attribute that can functionally determine the other attributes as well we also get these dependencies.

$$Ptal \longrightarrow Per_id, fingardag, kyn$$

$$Ptal \longrightarrow Per_id \longrightarrow fornavn$$

$$Ptal \longrightarrow Per_id \longrightarrow efturnavn$$

$$Ptal \longrightarrow Ptal$$

This shows us that both Ptal and Per_id are super keys and more specifically both are candidate keys. Since [1] tells us that when all attributes in a table depend on the superkey then we have a table in Boyce-Codd Normal Form. Then we can say that the person table we have is in Boyce-Codd Normal Form. Now looking at the loggur table we can also determine that the table is in Boyce-Codd Normal Form since

$$log_id \longrightarrow saldo_broyting$$

$$log_id \longrightarrow date$$

$$log_id \longrightarrow egin_tekstur$$

$$log_id \longrightarrow leypandi_saldo$$

And since all attributes are dependent on the candidate key we have the table in Boyce-Codd Normal form.

5 Implementation of the Database

The database was implemented using Oracle SQL. We started by creating tables corresponding to figure 1 and setting primary key and foreign key constraints on each table. The constraints mentioned in the database design section were then also implemented.

In order to implement the p-tal and validate that every p-tal followed the Faroese standard we made a function. This function took as inputs a birthday and a gender initial, "m" for male and "k" for female. The function then calculates a potential p-tal and checks if someone already has the same one. If no one has the calculated p-tal then it is returned as valid for use. In order to make sure that no one mistakenly inserts a wrong p-tal a trigger has been made on inserts for the table. Whenever before an insert was executed on the table the trigger would instead call the p-tal generator and use the generated p-tal instead of anything the user might have written. This ensured p-tals were always in the correct format.

P-tals have a specific format the need to adhere to. This format can be described by this equation.

$$\text{mod}(3x_1 + 2x_2 + 7x_3 + 6x_4 + 5x_5 + 4x_6 + 3x_7 + 2x_8 + x_9, 11) = 0$$

Where x_1, x_2 are the day of birth, x_3, x_4 are the month of birth, x_5, x_6 are the last two digits of the year of birth, x_7 can be between 0-4 for odd numbered centuries and between 5-9 for even numbered centuries. x_8 is an iterator that is used together with x_9 to make sure the sum is divisible by 11. x_9 is also odd for males and even for females.

All people are stored in the **pers** table regardless of whether they are customers, staff, or another type of person in the system. If a row is to be added to any of the other related tables, that person must already have a valid pre-existing row in the **pers** table.

We will describe the tables and their related procedures and views in the following order: the **pers** table, **hjúnaband**, **børn**, **starvsfolk**, **kundi**, and finally **bústað**.

Pers table:

We chose to make an identity primary key called **p_id**, which automatically generates a new value on insert. This was mainly done for convenience, ease of use, and to keep the design simple, since it removes the need for a separate sequence to generate a valid primary key.

In the **pers** table we also have a candidate key, namely **p_tal**, which has already been explained in greater detail earlier, so we will not repeat the full explanation here. In the implementation, **p_tal** was given the **UNIQUE** and **NOT NULL** constraints.

For the birthday attribute, we chose to use the standard **VARCHAR2** type and make it **NOT NULL**. We also put a default birthday value of '01010000' as a placeholder in order to avoid null handling. Our reasoning for choosing **VARCHAR2** instead of the **DATE** datatype was that it is easier to manipulate strings than dates when only parts of the birthday need to be accessed, for example by using substrings. Additionally, for the birthday attribute we added a check constraint using the regex format $\text{^[0-9]\{8\}\$}$. This means that the value must consist only of digits from 0-9, must be exactly 8 digits long, and must end immediately after the eighth digit.

We also have standard first name and last name attributes, both stored as `VARCHAR2(40)`. The only foreign key in the `pers` table is `bústað_id`.

We implemented a basic stored procedure for creating a new person, called `new_per`. It uses basic error handling through `IF` statements, and if the input passes the checks, it inserts a new row into the `pers` table. as person

Hjúnaband table:

The `hjúnaband` table can be described as an associative relationship table that represents the marital status between two people. It stores the two related persons, the date the marriage started, and the date it ended if the marriage has been dissolved. We also created a view which, when the marriage is active, gives each spouse access to view the other partner's accounts. This view is explained later.

We use an identity primary key called `hjúnaband_id`. The reasoning for this is the same as in the `pers` table section, so it will not be repeated here.

The table has two foreign keys, `p1_id` and `p2_id`, which relate to two different people in the `pers` table. These foreign keys are also set as `NOT NULL`. We have a check constraint that makes sure `p1_id` cannot be equal to `p2_id`. We also have a check constraint that enforces `p1_id < p2_id`. The reasoning for this is explained in the stored procedure section.

`gift_dato` and `skild_dato` comes next, both of which are of type `DATE`, with `gift_dato` having `SYSDATE` as its default value. A marriage is considered active if `skild_dato` is `NULL`. We also have a check constraint that ensures `skild_dato` cannot be before `gift_dato`.

We have two stored procedures for `hjún`, namely `nýggj_hjún` and `end_hjún`. The stored procedure `nýggj_hjún` is used for creating new marriages, while `end_hjún` is used for ending them.

`nýggj_hjún`:

This procedure works by taking two parameters, which are the individual `p_id` values of the two people being married, together with local variables. The procedure uses basic error handling through `IF` statements. It makes sure that a person cannot marry themselves, and it also checks whether any of the parameters are `NULL`. If they are, an error is raised.

To prevent duplicate marriages, we implemented the following solution:

```
p1_id := LEAST(p1_p_id, p_p2_p_id);  
p2_id := GREATEST(p1_p_id, p_p2_p_id);
```

We realised that if the ids are stored in the format `(p1_p_id, p2_p_id)`, and another marriage is then created with the same ids but in reversed order as `(p2_p_id, p1_p_id)`, the database would treat them as two different marriages. Our solution addresses this problem by forcing all ids into a fixed order where the smaller id is stored as `p1_id` and the greater id is stored as `p2_id`.

We then have checks to make sure that the persons exist and that neither person is already in an active marriage. Once these checks are completed, the procedure inserts the marriage as a new row where `skild_dato` is `NULL`, and the marriage is therefore active.

end_hjún:

The main function of this stored procedure is to end marriages by updating the attribute `skild_dato`. It uses the same basic error handling as the `nýggj_hjún` procedure. The main difference is that instead of inserting a new row, it uses an `UPDATE` statement to set `skild_dato = SYSDATE`. For error handling, it also uses a simple check to see whether an active marriage exists beforehand.

View between couples:

With this implementation, we created the view `v_hjunafelaga_kontur` to show which accounts a spouse is allowed to access. The view is based on the `hjúnaband` table and only includes active marriages, meaning rows where `skild_dato` IS NULL.

To make the relationship work for both spouses, the view uses `UNION ALL`. The first part returns rows where `p1_id` is the user and `p2_id` is the spouse, and the second part returns rows where `p2_id` is the user and `p1_id` is the spouse. This ensures that account visibility is symmetric. The result is then joined with `kundi` and `konto` so that the spouse's account ID and balance can be shown.

Børn table:

Similar to the `hjúnaband` table, the `børn` table is a relationship table that shows the connection between parent 1, an optional parent 2, and their child. The table has an identity primary key and three foreign keys: `p1_id`, `p2_id`, and `barn_id`. All three reference the `pers` table. However, only `p1_id` and `barn_id` are defined with the `NOT NULL` constraint. This means that in our design a child must have at least one parent and can have at most two.

In addition, we have constraints that ensure the child cannot also be one of its own parents, and that the same parent cannot be entered twice. We also gave `barn_id` a `UNIQUE` constraint, which means that the same child can only appear once in the table. This allows `barn_id` to be treated as a candidate key in this relation.

barn:

We have one stored procedure and one view that interact with the table. The stored procedure for `børn` is called `barn`, and its purpose is to insert rows into the `børn` table. It uses conditional statements to check that the data entered is correct and that the child does not already exist in the table. Since parent 2 is optional in our design, the procedure also checks whether the parameter for parent 2 is NULL or not. If parent 2 is given, it validates that this is also a correct `p_id`. After the validation checks are completed, the procedure inserts one row containing parent 1, optional parent 2, and the child.

Parent to child view:

The view that allows parents to see their child's account is called `v_foreldur_barn_kontur`. Its purpose is to show which accounts a parent is allowed to access for their child, and not in reverse, so the direction only goes from parent to child.

The view is based on the relationships stored in `børn`, and it works for both parent 1 and parent 2 by using `UNION ALL`. The first part of the view returns rows where `p1_id` is treated as the parent, and the second part returns rows where `p2_id` is treated as the

parent, as long as parent 2 is not NULL. In this way, both parents can be matched to the same child.

After this, the view joins with `pers`, `kundi`, and `konto`. The join with `pers` is used to identify the child, the join with `kundi` is used to connect the child to the banking part of the system, and the join with `konto` is used to return the child's account number and balance. This also means that the view only returns rows if the child is registered as a customer and already has an account in the system. If no matching customer row or account row exists, then no result is returned for that child.

The view also includes a condition that only shows these accounts while the child is under 18 years old. This is done by converting the child's `føðingardag` into a date, adding 18 years to it, and then checking whether that date is still later than the current date. Because of this, the parent can only view the child's accounts while the child is still considered a minor in the system.

Starvsfolk table:

The `starvsfolk` table is one of the more distinct tables in our system, since it is a complete subtype of the `pers` table and therefore does not have its own separate identity key. The primary key for `starvsfolk` is `p_id`, which means that its primary key is also a foreign key referencing the `pers` table. It stores only people who are employees of the bank. The other attributes are `starv_navn`, which is the job title and is set as NOT NULL, `lon`, which is salary, and `atgongd_typa`, which is stored as VARCHAR2 and has the default string value 'ONEYDUGT'.

The attribute `atgongd_typa` is the access type that allows staff members to perform different actions depending on their authorisation level. There are three authorisation levels, represented by the strings 'ONEYDUGT', 'STARVSFOLK', and 'ADMIN'. 'ONEYDUGT' is the default value and is intended for non-essential staff members who should not have access to the internal systems, such as janitors, housekeeping, and similar roles. The next level of authorisation is 'STARVSFOLK', which allows staff members to confirm or deny pending bookings. The final access type is 'ADMIN', which can do everything 'STARVSFOLK' can do, but in addition can create or update staff roles and change salary through stored procedures. We also have a check constraint that ensures that only these three strings can be used.

One thing to note regarding `lon` is that we considered it outside the scope of the task to make a functional salary system where staff are actually paid. In our implementation it is therefore simply stored as a number, and the salary is not added to the staff member's account.

We have four stored procedures that involve staff, namely `new_starv`, `rolle_broytari`, `boka_kladdu`, and `avvisa_kladdu`. Here we will focus on `new_starv` and `rolle_broytari`, and return to the other two later in the booking section.

`new_starv`

This procedure is used for creating a staff role.

It takes the admin's `p_id`, the `p_id` of the person being added, the title, and the salary as parameters. It first verifies that the admin has 'ADMIN' access before continuing, and then checks that the person exists before inserting them into the `starvsfolk` table. Their

default access type will always be 'ONEYDUGT', and the other procedure must be used if this is to be changed.

rolle_broytari

This procedure is used to update a pre-existing row in the `starvsfolk` table. It can change `starv_navn`, `lon`, and `atgongd_typa`. It works by only updating the parameters that are not NULL. For example, if `starv_navn` and `lon` are NULL, but `atgongd_typa` is 'STARVSFOLK', then `starv_navn` and `p_lon` remain unchanged and only `atgongd_typa` is updated in the row.

Similar to `new_starv`, it takes the `p_id` of the admin and checks whether they have the appropriate access level. It then checks whether the employee exists and updates the relevant row.

For accounts it was important that every account number is consistent with the Faroese standard for account numbers. This means that the first four numbers are the bank id, the next 3 numbers are the account type And the last 4 are used to uniquely identify the specific account. In a similar manner to how we handled the p-tal, we implemented a trigger that each time a record is inserted into the `konto` table then the account number generator function is called. In order to preserve uniqueness we used five different sequences. One for each account type we were testing with. The sequence would continuously generate a new number until a number fulfilling the requirements was generated. It would then return the account number as a string and the trigger would insert the generated account number instead of what anyone else might have tried to insert.

Kundi

The `kundi` table stores the customer specific part of a person. In our system, every customer must first exist as a row in `pers`, but not every person is necessarily a customer. The table therefore works as an extension of `pers` for people who are registered as banking customers.

The table contains a unique `kunda_id`, a reference to the related person through `p_id`, and attributes such as creation date, deletion date, and password. This makes it possible to separate general person information from the information that only applies to customers in the banking system.

The `kundi` table is also important because accounts are connected to customers rather than directly to persons. This means that many parts of the database, including views and transaction procedures, move through the chain `pers` -> `kundi` -> `konto` in order to connect a person to their accounts.

Account

The account number has the following requirement when being chosen.

$$\text{mod}(5x_1 + 4x_2 + 3x_3 + 2x_4 + 7x_5 + 6x_6 + 5x_7 + 4x_8 + 3x_9 + 2x_{10} + x_{11}, 11) = 0$$

Where x_1, x_2, x_3, x_4 are the registration number for the specific bank. We then chose x_5, x_6, x_7 to be the identifier for the account type. x_8, x_9, x_{10} are iterable numbers found

when creating a new account and finally x_{11} is the control number used to make the division by 11 fit.

An important note on how we implemented the booking system for the database is that it all goes through the `loggur` table. Meaning when a transfer or other account related change in balance were to happen then it would first be inserted into the `loggur` table where a trigger would then take the new record and use it's values to update the related account. This ensured every transaction is logged and that the account remains consistent with the log.

The draft and booking system are intertwined and in the following paragraphs we will describe the implementation of the draft system.

There are three types of drafts one can make, which are controlled by the `slag` attribute. They are represented by the following strings: `'INNSETING'`, `'UTTOKA'`, and `'FLYTING'`. We have a check constraint that makes sure it can only be these three strings. These three have different ways of operating.

All drafts are handled by the `nyggj_kladdu` stored procedure.

INNSETING: is when money is deposited into the account where `til_id` is required, but `frá_id` has to be `NULL`, and when booked a positive log will be added to the account.

UTTOKA: is the complete opposite of **INNSETING**. You can only use `frá_id` and `til_id` must be `NULL`, and when booked it will add a negative log row to the source account. The only additional part is that it will check if the source account has enough saldo before withdrawing.

FLYTING: is when money moves from one account in the bank to another account in the bank. It requires both `frá_id` and `til_id` and they cannot be the same account, and when booked, the source account will receive a negative log and the destination account will receive a positive log. Like **UTTOKA**, it will check if the source account has enough saldo to make the transfer before committing the change.

When a draft has been made, the attribute called `status` has the default string `'OAVGJORD'`, and applies it to all types of drafts, which will be used in the booking procedures.

Now we move on to the booking procedures.

Similar to the draft, we have three strings that determine the status of a booking, which is done by the attribute `status`. The three strings are `'OAVGJORD'`, `'BOKAD'` and `'AVVIST'`. As mentioned earlier, `'OAVGJORD'` is set as the default string for all drafts and we have a check constraint that makes sure that `status` can only be these three strings.

All bookings are handled by two stored procedures, `boka_kladdu` and `avvisa_kladdu`. All booking procedures can only be carried out by a staff member with the access type `'STARVSFOLK'` or `'ADMIN'`.

boka_kladdu:

If the staff member decides to approve the transaction, this procedure carries out the draft and updates the appropriate rows. It does this in the following way:

It reads the draft from `kladda` and checks if `status = 'OAVGJORD'`. If it is, it carries on if not it stops the procedure before any inserts have been made.

It then carries out the real transaction by inserting the values into `loggur` and updates

the draft status to 'BOKAD'.

avvisa_kladdu:

This procedure is activated when the staff member decides to reject the draft.

It changes the draft status to 'AVVIST', preventing it from being booked again, and it does not insert any transaction into `loggur` and does not move money in any way.

The banking system should also be able to handle calculating interest. We accomplished this by combining a couple of different techniques. First we used the `kontoslag` table to store important constants for different account types. To calculate the interest for one account we have made a function called 'rentu_rokning' that takes as inputs an account id, account type, start date and end date. Where start and end date are the dates you want to calculate interest from and to. Interest is then calculated daily but only added to the balance monthly. This means that bookings in the middle of your start and end date are accounted for in the calculation. Then we use a procedure that creates a cursor over all accounts and feeds them into the interest calculating function. The procedure then books all the interest calculations in the `loggur` table. We then implemented a scheduler that would run this procedure on the first day of the month calculate the interest accumulated over the previous month.

Bústað:

Stores address information such as street, house number, floor, room number, and a reference to the postal code. It is connected to pers through `bústað_id`, which means a person gets their address through this table. Since addresses are stored separately, several persons can share the same address if needed.

Postkota:

The postkota table stores postal code information, consisting of the postal code and the settlement name. It is separated from `bústað` in order to avoid repeated storage of the same postal area data and to support normalization of the database design.

Aggregate view:

We only have one aggregate view, called `v_kundi_samla_saldo`. It starts from the `kundi` table, joins each customer to their accounts in `konto`, and then groups the result by customer.

The view returns the customer id, the related person id, how many accounts the customer has, and the total balance across all of that customer's accounts. This is useful because it allows the customer to see the full balance across all of their accounts instead of having to check them one by one.

It is an aggregate view because it uses the aggregate functions `COUNT` and `SUM`.

5.1 The GUI

The main focus of the project was database design and SQL implementation, we considered it reasonable to use AI assistance in the development of the GUI. The graphical interface was therefore treated as a supporting layer around the database solution, while the

main academic part of the project remained the schema design, constraints, procedures, triggers, and views implemented in SQL. The GUI can be found in the appendix in the github link, the relevant files are bank_db.py and bank_gui.py

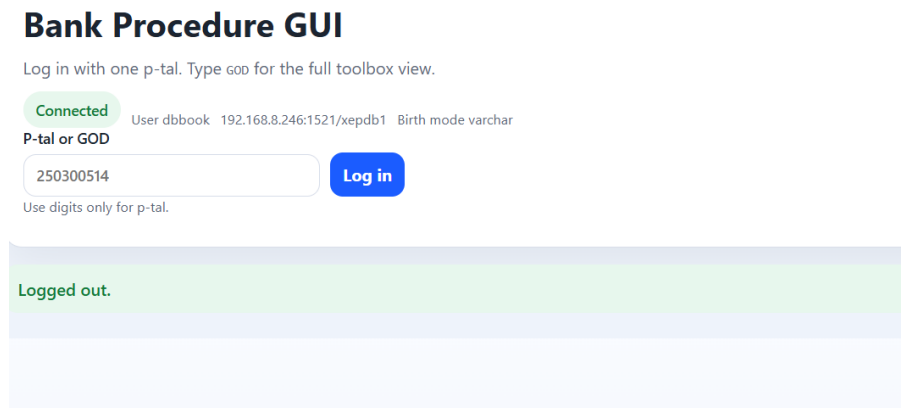


Figure 2: This is a picture of the login screen for our GUI

The GUI was implemented as a web-based interface directly connected to the database. As shown in the screenshot above, the user first logs in by entering a **p_tal**, while writing **GOD** gives full administrative access to the system, beyond admin which allows you to interact with every table, including the ones staff admin access. After login, the interface shows the functions available for that specific user role. This means that a normal user only sees their own information, while staff members and administrators are given access to additional functions.

LOG_ID	KONTO_ID	EIGARI_P_ID	EIGARI_NAVN	LOG_DATO	SALDO_BROYTING	LEYPAIDI_SALDO	MOTTAKARI_KONTO_ID	MOTTAKARI_P_ID	MOTTAKARI_NAVN	TEKST	NUVERAND
10	69692002229	2	Maria Joensen	2026-03-26 09:07:08	5000.0	5000.0					5000.0
9	69692002245	3	Páll Olsen	2026-03-26 06:44:37	0.0					Konto stovnað	0.0
8	69692002229	2	Maria Joensen	2026-03-26 06:44:14	0.0					Konto stovnað	5000.0
7	69690000016	5	Rob Banks	2026-03-26 06:30:04	0.0					Konto stovnað	1000000000
6	69694001629	1	Jón Hansen	2026-03-	0.0					Konto	0.0

Figure 3: Kontuavrit

As shown in figure 3, the user can open the account statement section in order to view **kontuavrit**. This gives an overview of the user's account activity and saldo based on the corresponding database view.

Figure 4: kassakladdu

In the same way, the GUI also makes it possible to enter a new kassakladdu, where the user fills in the transaction details and creates a draft row in `kladda`. The transaction is not carried out immediately, but must later be approved and booked by staff or an administrator.

6 Discussion

Creating a full banking database that can handle all the real world problems that might arise is an immense task and outside the scope of this project. But given enough time there would be many interesting edge cases that should be handled. Currently the database can only handle people owning account but not abstract entities like a bank or a business. Our work around was to just create a fictional person that owned all the banking accounts. But if we were to make a system that could handle business accounts then we would probably make another table called business accounts which would be related to a business table.

Another important aspect we didn't do as much with as we would like was the checking of some extra integrity constraints. For example a bank should never be able to create money out of thin air and the sum of all the accounts should always be some start value that the bank started with. Currently our `UTTOKA` and `INNSETING` just delete and add money to the system without subtracting it from somewhere else. We should probably have added a money supply account that when you deposit money into the bank then the money supply in the real world decreases because the money supply inside the bank increased. And vice versa the case where someone withdraws money from the bank.

The final important detail we have neglected in our implementation are security concerns. There has not been any extra energy used on making the system resistant against hostile actors. But this is a very important aspect that any real world banking system must deal with. If we were to have more time we would work on a proper login method in the GUI and write the GUI ourselves in order to make sure the AI didn't leave any glaring security issues. We would also connect to the database with a limited user instead of using only the admin user.

7 Conclusion

In conclusion we made an ER diagram of our banking system and implemented it into a sql database where we made it possible for customers to create accounts and make transactions with their accounts. A person could view the accounts of their spouse and children. And finally we made it possible to calculate interest on accounts and a GUI was made to view the contents of the database.

References

- [1] Abraham Silberschatz, Henry F. Korth & S. Sudarshan. (2020) *Database System Concepts Seventh Edition* Mc Graw Hill Education. ISBN 978-0-07-802215-9

8 Appendix

The link to the github used for this project:

https://github.com/Frystiboks/banking_student_project_database/tree/main

```
DROP TABLE loggur;  
DROP TABLE kladda;  
DROP TABLE starvsfolk;  
DROP TABLE brn;  
DROP TABLE hjnaband;  
DROP TABLE konto;  
DROP TABLE kundi;  
DROP TABLE kontoslag;  
DROP TABLE pers;  
DROP TABLE bsta;  
DROP TABLE postkota;
```

```
CREATE TABLE postkota (  
    postkota NUMBER PRIMARY KEY,  
    bygd VARCHAR(20)  
);
```

```
CREATE TABLE bsta (  
    bsta_id NUMBER PRIMARY KEY,  
    postkota NUMBER,  
    gta VARCHAR2(40),  
    hsnummar NUMBER,  
    hdd NUMBER,  
    rmnummar NUMBER,  
    CONSTRAINT fk_postkota  
        FOREIGN KEY (postkota) REFERENCES postkota(postkota)  
);
```

```

CREATE TABLE pers (
    p_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    p_tal VARCHAR2(11) UNIQUE NOT NULL,
    fornavn VARCHAR2(40),
    eftirnavn VARCHAR2(40),
    fððingardag VARCHAR2(8) DEFAULT '01010000' NOT NULL,
    kyn varchar2(1) DEFAULT 'm' NOT NULL,
    CONSTRAINT chk_kyn CHECK (kyn IN ('m', 'k')),
    bústað_id NUMBER,
    CONSTRAINT fk_bústað_id
        FOREIGN KEY (bústað_id) REFERENCES bústað(bústað_id),
    CONSTRAINT chk_fodingardag
        CHECK (REGEXP_LIKE(fððingardag, '~[0-9]{8}$'))
);

CREATE TABLE kundi (
    kunda_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    stovningar_dato date DEFAULT sysdate not null,
    slett_dato date,
    loynuorð varchar2(40),
    p_id number,
    CONSTRAINT fk_konto_eigari
        FOREIGN KEY (p_id) REFERENCES pers(p_id)
);

CREATE TABLE kontoslag (
    kontoslag_id VARCHAR2(3) PRIMARY KEY,
    slag_navn varchar2(40),
    credit_renta NUMBER(4,3),
    debit_renta NUMBER(4,3)
);

CREATE TABLE konto (
    konto_id VARCHAR2(11) DEFAULT '0' PRIMARY KEY,
    saldo NUMBER(12,2),
    kontotypa varchar2(3),
    eigari_p_id NUMBER,
    CONSTRAINT fk_konto_kundi
        FOREIGN KEY (eigari_p_id) REFERENCES kundi(kunda_id),
    CONSTRAINT fk_kontoslag
        FOREIGN KEY (kontotypa) REFERENCES kontoslag(kontoslag_id)
);

CREATE TABLE hjúnaband (
    hjúnaband_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    p1_id NUMBER NOT NULL,
    p2_id NUMBER NOT NULL,
    gift_dato DATE DEFAULT SYSDATE NOT NULL,

```

```

    skild_dato DATE,
    CONSTRAINT fk_hjúaband_p1
        FOREIGN KEY (p1_id) REFERENCES pers(p_id),
    CONSTRAINT fk_hjúaband_p2
        FOREIGN KEY (p2_id) REFERENCES pers(p_id),
    CONSTRAINT chk_hjúaband_ikki_sami
        CHECK (p1_id != p2_id),
    CONSTRAINT chk_hjúaband_rað
        CHECK (p1_id < p2_id),
    CONSTRAINT chk_hjúaband_dato
        CHECK (skild_dato IS NULL OR skild_dato >= gift_dato)
);

CREATE UNIQUE INDEX hjúaband_aktivt_par
    ON hjúaband (
        CASE WHEN skild_dato IS NULL THEN p1_id END,
        CASE WHEN skild_dato IS NULL THEN p2_id END
    );

CREATE TABLE starvsfolk (
    p_id number PRIMARY KEY,
    starv_navn VARCHAR2(20) NOT NULL,
    lon NUMBER,
    atgongd_typa VARCHAR2(20) DEFAULT 'ONEYDUGT' NOT NULL,

    CONSTRAINT fk_starvsfolk_p_id
        FOREIGN KEY (p_id) REFERENCES pers(p_id),

    CONSTRAINT chk_atgongd_typa
        CHECK (atgongd_typa IN ('ONEYDUGT', 'STARVSFOLK', 'ADMIN'))
);

CREATE TABLE börn (
    id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    p1_id NUMBER NOT NULL,
    p2_id NUMBER,
    barn_id NUMBER NOT NULL,
    CONSTRAINT fk_born_p1
        FOREIGN KEY (p1_id) REFERENCES pers(p_id),
    CONSTRAINT fk_born_p2
        FOREIGN KEY (p2_id) REFERENCES pers(p_id),
    CONSTRAINT fk_born_barn
        FOREIGN KEY (barn_id) REFERENCES pers(p_id),
    CONSTRAINT uq_born_barn
        UNIQUE (barn_id),
    CONSTRAINT chk_born_p1_barn
        CHECK (p1_id != barn_id),
    CONSTRAINT chk_born_p2_barn
        CHECK (p2_id IS NULL OR p2_id != barn_id),

```

```

        CONSTRAINT chk_born_p1_p2
            CHECK (p2_id IS NULL OR p1_id != p2_id)
    );

CREATE TABLE kladda (
    kladdu_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    flyting NUMBER(12,2) NOT NULL,
    frá_id VARCHAR2(11),
    til_id VARCHAR2(11),
    egintekst VARCHAR2(160),
    mottokutekst VARCHAR2(160),
    bokad_av_p_id NUMBER,
    bokad_dato DATE,
    slag VARCHAR2(20) NOT NULL,
    status VARCHAR2(20) DEFAULT 'OAVGJORD' NOT NULL,
    dato DATE DEFAULT SYSDATE NOT NULL,

    CONSTRAINT fk_kladda_fra
        FOREIGN KEY (frá_id) REFERENCES konto(konto_id),

    CONSTRAINT fk_kladda_til
        FOREIGN KEY (til_id) REFERENCES konto(konto_id),

    CONSTRAINT chk_kladda_flyting
        CHECK (flyting > 0),

    CONSTRAINT chk_kladda_slag
        CHECK (slag IN ('INNSETING', 'UTTOKA', 'FLYTING')),

    CONSTRAINT chk_kladda_status
        CHECK (status IN ('OAVGJORD', 'BOKAD', 'AVVIST')),

    CONSTRAINT chk_kladda_ikki_sama_konto
        CHECK (frá_id IS NULL OR til_id IS NULL OR frá_id != til_id),

    CONSTRAINT chk_kladda_slag_kontur
        CHECK (
            (slag = 'INNSETING' AND frá_id IS NULL AND til_id IS NOT NULL)
            OR
            (slag = 'UTTOKA' AND frá_id IS NOT NULL AND til_id IS NULL)
            OR
            (slag = 'FLYTING' AND frá_id IS NOT NULL AND til_id IS NOT NULL
             AND frá_id != til_id)
        )
);

```

```

CREATE TABLE loggur (
    log_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    konto_id VARCHAR2(11) NOT NULL,
    saldo_broyting NUMBER(12,2) NOT NULL,
    log_dato DATE DEFAULT SYSDATE NOT NULL,
    móttakari_id VARCHAR2(11),
    tekst VARCHAR2(160),
    leypanði_saldo number(12,2),
    CONSTRAINT fk_log_konto
        FOREIGN KEY (konto_id) REFERENCES konto(konto_id),
    CONSTRAINT fk_log_mottakari
        FOREIGN KEY (móttakari_id) REFERENCES konto(konto_id)
);

/

CREATE OR REPLACE FUNCTION ptal_gen(føðingardag IN VARCHAR2, kyn IN VARCHAR2)
RETURN VARCHAR2
IS
    summ NUMBER;
    I NUMBER;
    J NUMBER;
    rest NUMBER;
    j_end NUMBER;
    p_count NUMBER;
    new_ptal VARCHAR(9);
    kyn_tal NUMBER;
BEGIN
    IF LENGTH(føðingardag) != 8 THEN
        RETURN '';
    END IF;
    IF kyn = 'm' THEN
        kyn_tal := 1;
    ELSIF kyn = 'k' THEN
        kyn_tal := 0;
    END IF;

    IF MOD(TO_NUMBER(substr(føðingardag, 6, 1)),2) = 0 THEN
        J := 5;
    ELSE
        J := 0;
    END IF;
    j_end := J +4;
    I := 0;

    LOOP
        LOOP

```

```

new_ptal := NULL;
p_count := NULL;

summ :=
3*TO_NUMBER(substr(føðingardag, 1, 1)) +
2*TO_NUMBER(substr(føðingardag, 2, 1)) +
7*TO_NUMBER(substr(føðingardag, 3, 1)) +
6*TO_NUMBER(substr(føðingardag, 4, 1)) +
5*TO_NUMBER(substr(føðingardag, 7, 1)) +
4*TO_NUMBER(substr(føðingardag, 8, 1)) +
3*J +
2*I +
1*0;

rest :=11- MOD(summ, 11);
IF rest < 10 AND MOD(rest, 2) = kyn_tal THEN
    new_ptal := substr(føðingardag, 1, 4) ||
    substr(føðingardag, 7, 8) || to_char(J) || to_char(I)||
    to_char(rest);
    --return rest;
    SELECT COUNT(*)
    INTO p_count
    FROM pers
    WHERE p_tal = new_ptal;
END IF;
I := I+1;
EXIT WHEN I > 9 OR (p_count = 0 AND new_ptal IS NOT NULL);
END LOOP;
I := 0;
J := J+1;
EXIT WHEN J > j_end OR (p_count = 0 AND new_ptal IS NOT NULL);
END LOOP;
RETURN new_ptal;
END;

```

/

```

create sequence kontonum_seq_lán;
create sequence kontonum_seq_uppsparing;
create sequence kontonum_seq_nýtsla;
create sequence kontonum_seq_default;
create sequence kontonum_seq_bank;

create or replace function kontonummar_gen(kontotypa IN varchar2)
return varchar2
is
    summ number;

```

```

    i number;
    j number;
    k number;
    x number;
    abc varchar2(3);
    rest number;
    new_kontonummar varchar(11);
    max_kontonummar varchar(11);
begin

if kontotypa = '100' then
    abc := '100';
    select kontonum_seq_lán.nextval into x from dual;
elsif kontotypa = '200' then
    abc := '200';
    select kontonum_seq_nýtsla.nextval into x from dual;
elsif kontotypa = '300' then
    abc := '300';
    select kontonum_seq_uppsparing.nextval into x from dual;
elsif kontotypa = '000' then
    abc := '000';
    select kontonum_seq_bank.nextval into x from dual;
else
    abc := '400';
    select kontonum_seq_default.nextval into x from dual;
end if;

i := mod(x,10);
j := mod(floor(x/10),10);
k := mod(floor(x/100),10);

new_kontonummar := null;
if x < 1000 then
loop
    summ :=
        5*6 +
        4*9 +
        3*6 +
        2*9 +
        7*to_number(SUBSTR(abc, 1, 1)) +
        6*to_number(SUBSTR(abc, 2, 1)) +
        5*to_number(SUBSTR(abc, 3, 1)) +
        4*k +
        3*j +
        2*i +

```

```

1*0;

rest :=11- mod(summ, 11);
--return rest;
if rest < 10 then
    new_kontonummar := '6969' || abc || to_char(k) || to_char(j) ||
    to_char(i)|| to_char(rest);
end if;

if kontotypa = '100' then
    select kontonum_seq_lán.nextval into x from dual;
elsif kontotypa = '200' then
    select kontonum_seq_nýtsla.nextval into x from dual;
elsif kontotypa = '300' then
    select kontonum_seq_uppsparing.nextval into x from dual;
elsif kontotypa = '000' then
    select kontonum_seq_bank.nextval into x from dual;
else
    select kontonum_seq_default.nextval into x from dual;
end if;
i := mod(x,10);
j := mod(floor(x/10),10);
k := mod(floor(x/100),10);
exit when x> 999 or new_kontonummar is not null;
end loop;

end if;

return new_kontonummar;
end;
/

CREATE OR REPLACE PROCEDURE new_per (
    fornavn      IN VARCHAR2,
    eftirnavn    IN VARCHAR2,
    fððingardag  IN VARCHAR2,
    kyn          IN VARCHAR2,
    bústað_id    IN NUMBER
) IS
BEGIN
    IF fððingardag IS NULL OR LENGTH(fððingardag) <> 8 THEN
        RAISE_APPLICATION_ERROR(-20001, 'ERROR: Fððingardagur má vera DDMMYYYY.');
```

```

    END IF;

    IF kyn NOT IN ('m', 'k') THEN
        RAISE_APPLICATION_ERROR(-20002, 'ERROR: Kyn má vera m ella k.');
```



```

IF ptal_gen(føðingardag, kyn) IS NULL THEN
    RAISE_APPLICATION_ERROR(-20003, 'ERROR: Ógildugur føðingardagur.');
```

END IF;

```

INSERT INTO pers(fornavn, eftirnavn, føðingardag, kyn, bústað_id)
VALUES (fornavn, eftirnavn, føðingardag, kyn, bústað_id);
```

EXCEPTION

```

    WHEN OTHERS THEN
        RAISE;
```

END;

/

```

CREATE OR REPLACE PROCEDURE barn (
    p1_p_id    IN NUMBER,
    p2_p_id    IN NUMBER,
    barn_p_id  IN NUMBER
) IS
    dummy NUMBER;
```

BEGIN

```

    IF p1_p_id IS NULL OR barn_p_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20006, 'Foreldur 1 og barn mugu veljast.');
```

END IF;

```

    SELECT p_id
        INTO dummy
        FROM pers
        WHERE p_id = p1_p_id;
```

```

    SELECT p_id
        INTO dummy
        FROM pers
        WHERE p_id = barn_p_id;
```

```

    IF p1_p_id = barn_p_id THEN
        RAISE_APPLICATION_ERROR(-20001, 'Foreldur 1 kann ikki vera sami
        persónur sum barnið.');
```

END IF;

```

    IF p2_p_id IS NOT NULL THEN
        SELECT p_id
            INTO dummy
            FROM pers
            WHERE p_id = p2_p_id;
```

```

        IF p2_p_id = p1_p_id THEN
            RAISE_APPLICATION_ERROR(-20002, 'Foreldur 1 og foreldur 2
```

```

        kunnu ekki vera sami persónur.');
```

END IF;

```

    IF p2_p_id = barn_p_id THEN
        RAISE_APPLICATION_ERROR(-20003, 'Foreldur 2
        kann ekki vera sami persónur sum barnið.');
```

END IF;

END IF;

```

INSERT INTO börn (p1_id, p2_id, barn_id)
VALUES (p1_p_id, p2_p_id, barn_p_id);
```

EXCEPTION

```

    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'Ein av persónunum er ekki til í pers.');
```

WHEN DUP_VAL_ON_INDEX THEN

```

        RAISE_APPLICATION_ERROR(-20005, 'Barnið hefur longu eina familju-røð.');
```

WHEN OTHERS THEN

```

        RAISE;
```

END barn;

/

```

CREATE OR REPLACE FUNCTION rentu_rokning(kontotypa IN VARCHAR2,
p_konto_id IN VARCHAR2,
dato_start IN DATE, dato_end IN DATE)
RETURN NUMBER
IS
    saldo NUMBER(12,2);
    last_dato DATE;
    max_dato DATE;
    dato_current DATE;
    end_of_month DATE;
    renta NUMBER;
    samla_renta NUMBER;
    debit_renta NUMBER;
    credit_renta NUMBER;
    max_trans_id NUMBER;
BEGIN

SELECT credit_renta INTO credit_renta
FROM kontoslag
WHERE kontotypa = kontoslag.kontoslag_id;

SELECT debit_renta INTO debit_renta
FROM kontoslag
WHERE kontotypa = kontoslag.kontoslag_id;

    samla_renta := 0;
```

```

SELECT MAX(log_dato) INTO max_dato
FROM loggur
WHERE p_konto_id = loggur.konto_id AND log_dato <= dato_start;

-- makes sure we get the latest transaction in the case
--where we have duplicate dates.
SELECT MAX(log_id) INTO max_trans_id
FROM loggur
WHERE p_konto_id = loggur.konto_id AND log_dato = max_dato;

-- loys trupuleikan um eingin bóking er.
--kanska ger eina 0 bóking tá kontoin er stovna.
SELECT leypanði_saldo INTO saldo
FROM loggur
WHERE p_konto_id = loggur.konto_id AND log_dato = max_dato
AND max_trans_id = loggur.log_id;

SELECT last_day(dato_start) INTO end_of_month FROM dual;
dato_current := dato_start;

LOOP
IF saldo < 0 THEN
    renta := credit_renta/364.25;
ELSE
    renta := debit_renta/364.25;
END IF;
samla_renta := samla_renta + saldo*renta;

SELECT MAX(log_id) INTO max_trans_id
FROM loggur
WHERE p_konto_id = loggur.konto_id AND log_dato >= dato_current
AND log_dato < dato_current + 1;

IF max_trans_id IS NOT NULL THEN
    SELECT leypanði_saldo INTO saldo
    FROM loggur
    WHERE p_konto_id = loggur.konto_id AND max_trans_id = loggur.log_id;
END IF;

IF dato_current = end_of_month THEN
    saldo := saldo + samla_renta;
    samla_renta := 0;
    SELECT last_day(add_months(dato_current, 1)) INTO end_of_month FROM dual;
END IF;

```

```

        EXIT WHEN dato_current >= dato_end;
        SELECT dato_current + 1 INTO dato_current FROM dual;
    END LOOP;
RETURN saldo;
END;
/

CREATE OR REPLACE PROCEDURE rentu_rokning_allar_konti(dato_start IN DATE,
dato_end IN DATE)
IS
    rentu_saldo NUMBER;
    bankaboks VARCHAR2(11);
BEGIN
    bankaboks := '69690000016';
    FOR rec IN (
        SELECT konto_id, kontotypa, saldo
        FROM konto
    ) LOOP
        SELECT rentu_rokning(rec.kontotypa, rec.konto_id, dato_start, dato_end)
        INTO rentu_saldo FROM dual;

        INSERT INTO loggur (konto_id, saldo_broyting, móttakari_id)
        VALUES (rec.konto_id, rentu_saldo - rec.saldo, bankaboks);

        INSERT INTO loggur (konto_id, saldo_broyting, móttakari_id)
        VALUES (bankaboks, -(rentu_saldo - rec.saldo), rec.konto_id);

    END LOOP;
END;
/

CREATE OR REPLACE PROCEDURE new_starv (
    brukari_p_id    IN NUMBER, -- Er admin sum loggar inn
    p_id            IN NUMBER,  -- er nýggji starvsfólk
    starv_navn      IN VARCHAR2,
    lon             IN NUMBER
) IS
    brukari_atgongd starvsfolk.atgongd_typa%TYPE;
    dummy           NUMBER;
BEGIN
    IF lon <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'ERROR: Løn má vera yvir 0');
    END IF;

    SELECT atgongd_typa
    INTO brukari_atgongd
    FROM starvsfolk
    WHERE p_id = brukari_p_id;

```

```

IF brukari_atgongd <> 'ADMIN' THEN
    RAISE_APPLICATION_ERROR(-20002, 'ERROR:
    Bert administrator kann stovna starvsfólk.');
```

END IF;

```

SELECT p_id
INTO dummy
FROM pers
WHERE p_id = p_id;

INSERT INTO starvsfolk (starv_navn, lon, p_id)
VALUES (starv_navn, lon, p_id);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20003, 'ERROR: Annaðhvørt er aktørurinn ikki
        starvsfólk, ella persónurinn er ikki til.');
```

WHEN DUP_VAL_ON_INDEX THEN

```

    RAISE_APPLICATION_ERROR(-20004, 'ERROR: Hesin persónur er longu
    skrásettur sum starvsfólk.');
```

WHEN OTHERS THEN

```

    RAISE;
```

END;

/

-- Rolle broytari er fyri at broyta rolluna hjá starvfólkum,
-- har ein admin kann broyta løn,
-- Starv navn, access typa og meira

```

CREATE OR REPLACE PROCEDURE rolle_broytari (
    brukari_p_id IN NUMBER,
    p_id          IN NUMBER,
    starv_navn    IN VARCHAR2,
    lon           IN NUMBER,
    atgongd_typa  IN VARCHAR2
) IS
    brukari_atgongd  starvsfolk.atgongd_typa%TYPE;
    dummy            NUMBER;
```

BEGIN

```

    SELECT atgongd_typa
    INTO brukari_atgongd
    FROM starvsfolk
    WHERE p_id = brukari_p_id;

    IF brukari_atgongd <> 'ADMIN' THEN
        RAISE_APPLICATION_ERROR(-20002, 'ERROR: Bert
        administrator kann broyta starvsfólk.');
```

```

END IF;

SELECT p_id
INTO dummy
FROM starvsfolk
WHERE p_id = p_id;

IF lon IS NOT NULL THEN
    IF lon <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'ERROR: Løn má vera yvir 0.');
```

END IF;

```

END IF;

IF atgongd_typa IS NOT NULL THEN
    IF atgongd_typa NOT IN ('ONEYDUGT', 'STARVSFOLK', 'ADMIN') THEN
        RAISE_APPLICATION_ERROR(-20005, 'ERROR: Ógildig atgongd_typa.');
```

END IF;

```

END IF;

IF starv_navn IS NOT NULL THEN
    UPDATE starvsfolk
    SET starv_navn = starv_navn
    WHERE p_id = p_id;
END IF;

IF lon IS NOT NULL THEN
    UPDATE starvsfolk
    SET lon = lon
    WHERE p_id = p_id;
END IF;

IF atgongd_typa IS NOT NULL THEN
    UPDATE starvsfolk
    SET atgongd_typa = atgongd_typa
    WHERE p_id = p_id;
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20003, 'ERROR: Annaðhvørt finst broytarin ikki
        sum starvsfólk, ella finst starvsfólkið ikki.');
```

WHEN OTHERS THEN

```

    RAISE;
END rolle_broytari;
/

CREATE OR REPLACE PROCEDURE nýggj_hjún (
    p1_p_id IN NUMBER,
```

```

        p2_p_id IN NUMBER
    ) IS
        p1_id          NUMBER;
        p2_id          NUMBER;
        person_count NUMBER;
        active_count  NUMBER;
BEGIN
    IF p1_p_id IS NULL OR p2_p_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20000, 'Báðir persónar mugu veljast.');
```

END IF;

```

    IF p1_p_id = p2_p_id THEN
        RAISE_APPLICATION_ERROR(-20001, 'Ein persónur kann ekki giftast
        við sér sjálvum.');
```

END IF;

```

    p1_id := LEAST(p1_p_id, p2_p_id);
    p2_id := GREATEST(p1_p_id, p2_p_id);

    SELECT COUNT(*)
        INTO person_count
        FROM pers
        WHERE p_id IN (p1_id, p2_id);

    IF person_count <> 2 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Ein av persónunum er ekki til.');
```

END IF;

```

    SELECT COUNT(*)
        INTO active_count
        FROM hjúnaband
        WHERE skild_dato IS NULL
            AND (p1_id IN (p1_id, p2_id) OR p2_id IN (p1_id, p2_id));

    IF active_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Ein av persónunum er longu
        í virknu hjúnabandi.');
```

END IF;

```

    INSERT INTO hjúnaband (p1_id, p2_id, gift_dato, skild_dato)
    VALUES (p1_id, p2_id, SYSDATE, NULL);
END nýggj_hjún;
/

CREATE OR REPLACE PROCEDURE end_hjún (
    p1_p_id IN NUMBER,
    p2_p_id IN NUMBER
) IS
```

```

    p1_id          NUMBER;
    p2_id          NUMBER;
    person_count   NUMBER;
BEGIN
    IF p1_p_id IS NULL OR p2_p_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20000, 'Báðir persónar mugu veljast.');
```

END IF;

```

    IF p1_p_id = p2_p_id THEN
        RAISE_APPLICATION_ERROR(-20001, 'Ein persónur kann ekki
        skiljast frá sær sjálvum.');
```

END IF;

```

    p1_id := LEAST(p1_p_id, p2_p_id);
    p2_id := GREATEST(p1_p_id, p2_p_id);

    SELECT COUNT(*)
        INTO person_count
        FROM pers
        WHERE p_id IN (p1_id, p2_id);

    IF person_count <> 2 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Ein av persónunum er ekki til.');
```

END IF;

```

    UPDATE hjúnaband
        SET skild_dato = SYSDATE
        WHERE skild_dato IS NULL
            AND p1_id = p1_id
            AND p2_id = p2_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20005, 'Einki virkið hjúnaband funnið
        hjá hesum báðum.');
```

END IF;

END end_hjún;

/

```

CREATE OR REPLACE PROCEDURE byggj_kladda (
    brukari_p_id   IN NUMBER,
    flyting        IN NUMBER,
    fra_id         IN VARCHAR2,
    til_id         IN VARCHAR2,
    egintekst      IN VARCHAR2,
    mottokutekst   IN VARCHAR2,
    slag           IN VARCHAR2
) IS
    dummy NUMBER;
```



```

BEGIN
  IF brukari_p_id IS NULL THEN
    RAISE_APPLICATION_ERROR(-20001, 'ERROR: Brúkari má veljast.');
```

END IF;

```

  IF flyting IS NULL OR flyting <= 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'ERROR: Flyting má vera yvir 0.');
```

END IF;

```

  IF slag IS NULL OR slag NOT IN ('INNSETING', 'UTTOKA', 'FLYTING') THEN
    RAISE_APPLICATION_ERROR(-20003, 'ERROR: Ógildigt slag.');
```

END IF;

```

  SELECT p_id
  INTO dummy
  FROM pers
  WHERE p_id = brukari_p_id;
```

```

  IF slag = 'INNSETING' THEN
    IF til_id IS NULL OR fra_id IS NOT NULL THEN
      RAISE_APPLICATION_ERROR(-20004, 'ERROR: INNSETING krevur
      bert til-konto.');
```

END IF;

```

  SELECT 1
  INTO dummy
  FROM konto k
  JOIN kundi ku
    ON ku.kunda_id = k.eigari_p_id
  WHERE k.konto_id = til_id
    AND ku.p_id = brukari_p_id;
```

```

  ELSIF slag = 'UTTOKA' THEN
    IF fra_id IS NULL OR til_id IS NOT NULL THEN
      RAISE_APPLICATION_ERROR(-20005, 'ERROR: UTTOKA krevur bert frá-konto.');
```

END IF;

```

  SELECT 1
  INTO dummy
  FROM konto k
  JOIN kundi ku
    ON ku.kunda_id = k.eigari_p_id
  WHERE k.konto_id = fra_id
    AND ku.p_id = brukari_p_id;
```

```

  ELSIF slag = 'FLYTING' THEN
    IF fra_id IS NULL OR til_id IS NULL THEN
      RAISE_APPLICATION_ERROR(-20006, 'ERROR: FLYTING krevur
```

```

        bæði frá- og til-konto.');
```

END IF;

```

IF fra_id = til_id THEN
    RAISE_APPLICATION_ERROR(-20007, 'ERROR: Frá- og til-konto
    kunnu ekki vera eins.');
```

END IF;

```

SELECT 1
INTO dummy
FROM konto k
JOIN kundi ku
    ON ku.kunda_id = k.eigari_p_id
WHERE k.konto_id = fra_id
    AND ku.p_id = brukari_p_id;

SELECT 1
INTO dummy
FROM konto
WHERE konto_id = til_id;
END IF;

INSERT INTO kladda (
    flyting,
    frá_id,
    til_id,
    egintekst,
    mottokutekst,
    slag,
    status,
    dato
)
VALUES (
    flyting,
    fra_id,
    til_id,
    egintekst,
    mottokutekst,
    slag,
    'OAVGJORD',
    SYSDATE
);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20008, 'ERROR: Persónur ella konto fannst ekki,
        ella kontoin er ekki tín.');
```

WHEN OTHERS THEN

```

        RAISE;
END byggj_kladda;
/

CREATE OR REPLACE PROCEDURE boka_kladdu (
    brukari_p_id IN NUMBER,
    kladdu_id      IN NUMBER
) IS
    atgongd      starvsfolk.atgongd_typa%TYPE;
    flyting      kladda.flyting%TYPE;
    fra_id       kladda.fra_id%TYPE;
    til_id       kladda.til_id%TYPE;
    egintekst    kladda.egintekst%TYPE;
    mottokutekst kladda.mottokutekst%TYPE;
    slag         kladda.slag%TYPE;
    status       kladda.status%TYPE;
    fra_saldo    konto.saldo%TYPE;
    til_saldo    konto.saldo%TYPE;
BEGIN
    SELECT atgongd_typa
    INTO atgongd
    FROM starvsfolk
    WHERE p_id = brukari_p_id;

    IF atgongd != 'STARVSFOLK' AND atgongd != 'ADMIN' THEN
        RAISE_APPLICATION_ERROR(-20001, 'ERROR: Bert starvsfólk ella
        admin kunnu bóka.');
```

```

    END IF;

    SELECT flyting, frá_id, til_id, egintekst, mottokutekst, slag, status
    INTO flyting, fra_id, til_id, egintekst, mottokutekst, slag, status
    FROM kladda
    WHERE kladdu_id = kladdu_id
    FOR UPDATE;

    IF status != 'OAVGJORD' THEN
        RAISE_APPLICATION_ERROR(-20002, 'ERROR: Kladdan er longu viðgjørð.');
```

```

    END IF;

    IF slag = 'INNSETING' THEN
        SELECT saldo
        INTO til_saldo
        FROM konto
        WHERE konto_id = til_id
        FOR UPDATE;

        INSERT INTO loggur (
            konto_id,
```

```

        saldo_broyting,
        móttakari_id,
        tekst
    )
VALUES (
    til_id,
    flyting,
    NULL,
    mottokutekst
);

ELSIF slag = 'UTTOKA' THEN
    SELECT saldo
    INTO fra_saldo
    FROM konto
    WHERE konto_id = fra_id
    FOR UPDATE;

    IF flyting > fra_saldo THEN
        RAISE_APPLICATION_ERROR(-20003, 'ERROR: Upphædd er hægri enn saldo.');
```

```

    END IF;

    INSERT INTO loggur (
        konto_id,
        saldo_broyting,
        móttakari_id,
        tekst
    )
VALUES (
        fra_id,
        -flyting,
        NULL,
        egintekst
    );

ELSIF slag = 'FLYTING' THEN
    IF fra_id < til_id THEN
        SELECT saldo
        INTO fra_saldo
        FROM konto
        WHERE konto_id = fra_id
        FOR UPDATE;

        SELECT saldo
        INTO til_saldo
        FROM konto
        WHERE konto_id = til_id
        FOR UPDATE;
    
```

```

ELSE
    SELECT saldo
    INTO til_saldo
    FROM konto
    WHERE konto_id = til_id
    FOR UPDATE;

    SELECT saldo
    INTO fra_saldo
    FROM konto
    WHERE konto_id = fra_id
    FOR UPDATE;
END IF;

IF flyting > fra_saldo THEN
    RAISE_APPLICATION_ERROR(-20004, 'ERROR: Flyting má ekki
    vera yvir saldo.');
```

```

END IF;

INSERT INTO loggur (
    konto_id,
    saldo_broyting,
    móttakari_id,
    tekst
)
VALUES (
    fra_id,
    -flyting,
    til_id,
    egintekst
);

INSERT INTO loggur (
    konto_id,
    saldo_broyting,
    móttakari_id,
    tekst
)
VALUES (
    til_id,
    flyting,
    fra_id,
    mottokutekst
);

ELSE
    RAISE_APPLICATION_ERROR(-20006, 'ERROR: Ókent slag á kladdu.');
```

```

END IF;

```

```

UPDATE kladda
SET status = 'BOKAD',
    bokad_av_p_id = brukari_p_id,
    bokad_dato = SYSDATE
WHERE kladdu_id = kladdu_id;

COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20005, 'ERROR: Starvsfólk,
            kladda ella konto fannst ekki.');
```

WHEN OTHERS THEN

```

    RAISE;
END boka_kladdu;
/

CREATE OR REPLACE PROCEDURE avvisa_kladdu (
    brukari_p_id IN NUMBER,
    kladdu_id      IN NUMBER
) IS
    atgongd starvsfolk.atgongd_typa%TYPE;
BEGIN
    SELECT atgongd_typa
    INTO atgongd
    FROM starvsfolk
    WHERE p_id = brukari_p_id;

    IF atgongd != 'STARVSFOLK' AND atgongd != 'ADMIN' THEN
        RAISE_APPLICATION_ERROR(-20001, 'ERROR: Bert starvsfólk ella
            admin kunnu avvísa.');
```

END IF;

```

    UPDATE kladda
    SET status = 'AVVIST',
        bokad_av_p_id = brukari_p_id,
        bokad_dato = SYSDATE
    WHERE kladdu_id = kladdu_id
        AND status = 'OAVGJORD';

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'ERROR: Kladdan finst ekki
            ella er longu viðgjörð.');
```

END IF;

```

COMMIT;
```

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20003, 'ERROR: Starvsfólk fannst ekki.');
```

WHEN OTHERS THEN
 RAISE;
 END avvisa_kladdu;
 /

```

CREATE OR REPLACE VIEW v_foreldur_barn_kontur AS
SELECT
    rel.foreldur_id,
    rel.barn_id,
    k.konto_id,
    k.saldo
FROM (
    SELECT p1_id AS foreldur_id, barn_id
    FROM börn

    UNION ALL

    SELECT p2_id AS foreldur_id, barn_id
    FROM börn
    WHERE p2_id IS NOT NULL
) rel
JOIN pers barn
    ON barn.p_id = rel.barn_id
JOIN kundi ku
    ON ku.p_id = rel.barn_id
JOIN konto k
    ON k.eigari_p_id = ku.kunda_id
WHERE barn.føðingardag IS NOT NULL
    AND ADD_MONTHS(TO_DATE(barn.føðingardag, 'DDMMYYYY'), 12 * 18) > TRUNC(SYSDATE);
/

```

CREATE OR REPLACE VIEW v_hjunafelaga_kontur AS
 SELECT
 rel.brukari_p_id,
 rel.maki_p_id,
 k.konto_id,
 k.saldo
 FROM (
 SELECT p1_id AS brukari_p_id, p2_id AS maki_p_id
 FROM hjúnaband
 WHERE skild_dato IS NULL

 UNION ALL

 SELECT p2_id AS brukari_p_id, p1_id AS maki_p_id

```

        FROM hjúnaband
        WHERE skild_dato IS NULL
    ) rel
JOIN kundi ku
    ON ku.p_id = rel.maki_p_id
JOIN konto k
    ON k.eigari_p_id = ku.kunda_id;
/
CREATE OR REPLACE VIEW v_kontoavrit AS
SELECT
    l.log_id,
    l.konto_id,
    p.p_id AS eigari_p_id,
    p.fornavn || ' ' || p.eftirnavn AS eigari_navn,
    l.log_dato,
    l.saldo_broyting,
    l.leypandi_saldo,
    l.móttakari_id AS mottakari_konto_id,
    mp.p_id AS mottakari_p_id,
    mp.fornavn || ' ' || mp.eftirnavn AS mottakari_navn,
    l.tekst,
    k.saldo AS nuverandi_saldo
FROM loggur l
JOIN konto k
    ON k.konto_id = l.konto_id
JOIN kundi ku
    ON ku.kunda_id = k.eigari_p_id
JOIN pers p
    ON p.p_id = ku.p_id
LEFT JOIN konto mk
    ON mk.konto_id = l.móttakari_id
LEFT JOIN kundi mku
    ON mku.kunda_id = mk.eigari_p_id
LEFT JOIN pers mp
    ON mp.p_id = mku.p_id;
/
CREATE OR REPLACE VIEW v_kundi_samla_saldo AS
SELECT
    ku.kunda_id,
    ku.p_id,
    COUNT(k.konto_id) AS tal_av_kontum,
    SUM(k.saldo) AS samlað_saldo
FROM kundi ku
LEFT JOIN konto k
    ON k.eigari_p_id = ku.kunda_id
GROUP BY ku.kunda_id, ku.p_id;
/

```



```

-- allar kontoir mugu verða stovnaðar við saldo 0

create or replace trigger new_konto_trig_before before insert on konto
for each row
begin
    if :new.konto_id != '69690000016' then
        :new.konto_id := kontonummar_gen(:new.kontotypa);
    end if;
    :new.saldo := 0;
end;
/

create or replace trigger new_konto_trig_after after insert on konto
for each row
begin
    insert into loggur (konto_id,
                        saldo_broyting,
                        log_dato,
                        móttakari_id,
                        tekst,
                        leypandi_saldo
    )
        values (:new.konto_id,
                nvl(:new.saldo, 0),
                sysdate,
                null,
                'Konto stovnað',
                nvl(:new.saldo, 0)
        );
end;
/

create or replace trigger loggur_ins before insert on loggur
for each row
declare
    new_saldo number;
begin
    if :new.log_dato is null then
        :new.log_dato := sysdate;
    end if;
    new_saldo := 0;
    if :new.saldo_broyting != 0 then
        update konto
        set saldo = saldo + :new.saldo_broyting
        where konto_id = :new.konto_id;

        select saldo into new_saldo from konto where konto_id = :new.konto_id;
        :new.leypandi_saldo := new_saldo ;
    end if;
end;
/

```

```

        else
            :new.leypandi_saldo := :old.leypandi_saldo;
        end if;

end;
/

create or replace trigger new_pers_trig before insert on pers
for each row
begin
    :new.p_tal := ptal_gen(:new.føðingardag, :new.kyn);
end;
/

-- máðanarlig renturokning
begin
    -- sletta job um tað eksisterar
    begin
        dbms_scheduler.drop_job('mánaðarlig_renturokning', true);
    exception
        when others then
            if sqlcode != -27475 then
                raise;
            end if;
    end;

    dbms_scheduler.create_job (
        job_name          => 'mánaðarlig_renturokning',
        job_type           => 'plsql_block',
        job_action         => 'begin
                                rentu_rokning_allar_konti(
                                    add_months(trunc(sysdate), -1),
                                    trunc(sysdate));
                                end;',
        start_date         => systimestamp,
        repeat_interval    => 'freq=monthly;bymonthday=1;byhour=0;byminute=0;bysecond=0'
        enabled            => true
    );
end;
/

```